Plano De Testes - Compass ServeRest

1. Objeto de Testes

O objeto de testes é a API REST da compass: https://compassuol.serverest.dev/ .

2. Escopo

Os testes abrangerão todos os recursos presentes na API documentada no ServeRest, sendo os endpoints: /login (que gerencia a autenticação do usuário), /usuários (que gerencia os dados dos usuários), /produtos (gerencia os produtos) e /carrinhos (gerencia os carrinhos cadastrados pelos usuários).

3. Objetivos

- Verificar se a autenticação funciona corretamente na aplicação, se cada tipo de usuário tem suas respectivas permissões e a requisição de login.
- Verificar se as requisições que gerenciam os objetos “usuarios”, “produtos” e “carrinhos” manipulam corretamente os dados e retornam as respostas corretas.
- Analisar se a lógica de negócios da aplicação está sendo devidamente aplicada.

4. Mapeamento dos Endpoints

Endpoint	Verbo HTTP	Body (JSON)	Request Params	Função
/login	POST	email: string password: string	Nenhum	Deve ser retornado um token de autorização com a duração de 600 segundos.
/usuarios	GET	Nenhum	id: string nome: string email: string password: string administrator: string (true ou false)	Lista os usuários registrados.

/usuarios	POST	nome: string email: string password: string administrator: string (true ou false)	Nenhum	Cadastra um novo usuário. Não deve ser possível cadastrar um usuário com um email já cadastrado.
/usuarios/{id}	GET	Nenhum	id: string	Busca um usuário pelo id.
/usuarios/{id}	DELETE	Nenhum	id: string	Exclui um usuário. Não deve ser possível excluir um usuário com um carrinho.
/usuarios/{id}	PUT	nome: string email: string password: string administrator: string (true ou false)	id: string	Edita um usuário, caso não exista um com o id inserido, é criado um novo. Não deve ser possível cadastrar um usuário com um email já cadastrado.
/produtos	GET	Nenhum	id: string nome: string preco: integer descricao: string quantidade: integer	Lista os produtos cadastrados.
/produtos	POST	nome: string preco: integer descricao: string quantidade: integer	Nenhum	Cadastra um novo produto. Não deve ser possível criar um produto com um nome já utilizado.

/produtos/{id}	GET	Nenhum	id: string	Busca um produto pelo id.
/produtos/{id}	DELETE	Nenhum	id: string	Exclui um produto. Não deve ser possível excluir um produto dentro de um carrinho.
/produtos/{id}	PUT	nome: string preco: integer descricao: string quantidade: integer	id: string	Edita um produto. Caso não seja encontrado um produto com o id informado, é criado um novo. Não deve ser possível criar um produto com um nome já utilizado.
/carrinhos	GET	Nenhum	id: string precoTotal: integer quantidadeTotal: integer idUsuario: string	Lista os carrinhos cadastrados.
/carrinhos	POST	idProduto: string quantidade: integer	Token de autenticação	Cadastra um novo carrinho vinculado ao token do header. Só deve ser possível cadastrar um carrinho por usuário. É feita a redução da quantidade em /produtos a cada produto inserido no carrinho.

/carrinhos/{id}	GET	Nenhum	id: string	Busca um carrinho pelo id.
/carrinhos/concluir-compra	DELETE	Nenhum	Token de autenticação	Exclui um carrinho. O carrinho é vinculado ao token do usuário.
/carrinhos/cancelar-compra	DELETE	Nenhum	Token de autenticação	Exclui um carrinho. A quantidade dos produtos dentro do carrinho retorna a /produtos. O carrinho é vinculado ao token do usuário.

5. Riscos e Prioridades

Principais riscos identificados na API levando em consideração o grau de impacto no objetivo prioritário de vender produtos e fatores como a segurança do usuário, segurança da aplicação e cumprimento das regras de negócio:

- O usuário não conseguir efetuar o login ou não gerar token de autenticação;
- O usuário conseguir acessar um carrinho ou funções da administrador sem a devida autenticação;
- Impossibilidade de adicionar produtos ao carrinho;
- Erros ou indisponibilidade ao finalizar uma compra;
- Valores incorretos serem salvos no estoque;
- O usuário adquirir produtos fora de estoque.

6. Cenários de Teste

ID do Cenário	Endpoint	Verbo HTTP	Cenário	Resultado Esperado
TL-01	/login	GET	Realizar um login com dados válidos e usuário cadastrado.	A aplicação deve permitir o acesso e retornar o token.

TL-02	/login	GET	Realizar login com dados inválidos.	A aplicação deve retornar uma mensagem de erro indicando que os dados devem ser preenchidos corretamente.
TL-03	/login	GET	Realizar login com um email não cadastrado.	A aplicação deve retornar uma mensagem de erro informando que o usuário não está cadastrado.
TU-01	/usuarios	GET	Listar os usuários.	Retornar a lista de usuários.
TU-02	/usuarios	POST	Cadastrar um usuário com dados válidos.	Cadastrar o usuário e exibir uma mensagem de sucesso.
TU-03	/usuarios	POST	Cadastrar um usuário com dados inválidos.	Deverá ser retornada uma mensagem de erro indicando os campos que foram preenchidos incorretamente.
TU-04	/usuarios	POST	Cadastrar um usuário com um email já cadastrado.	Deverá ser exibida uma mensagem de erro indicando que o email informado já está cadastrado.
TU-05	/usuarios/{id}	GET	Buscar um usuário com um ID válido e existente.	A aplicação deve retornar o usuário correspondente ao ID informado.
TU-06	/usuarios/{id}	GET	Buscar um usuário com um ID inválido.	Exibir uma mensagem de erro informando que o ID informado está fora do formato correto.
TU-07	/usuarios/{id}	GET	Buscar um usuário com um ID inexistente.	Exibir uma mensagem de erro informando que o ID informado não foi encontrado.
TU-08	/usuarios/{id}	DELETE	Excluir um usuário com um ID válido e existente.	Retornar uma mensagem de sucesso e excluir o usuário correspondente do banco de dados.

TU-09	/usuarios/{id}	DELETE	Excluir um usuário com um ID inválido.	Exibir uma mensagem de erro informando que o ID informado está fora do formato correto.
TU-10	/usuarios/{id}	DELETE	Excluir um usuário com um ID inexistente.	Exibir uma mensagem de erro informando que o ID informado não foi encontrado.
TU-11	/usuarios/{id}	PUT	Atualizar o usuário, inserindo dados corretos e um ID existente.	Os dados do usuário devem ser atualizados e retornar uma mensagem de sucesso.
TU-12	/usuarios/{id}	PUT	Criar um novo usuário, inserindo dados corretos e um ID inexistente.	O novo usuário deve ser criado e retornar uma mensagem de sucesso.
TU-13	/usuarios/{id}	PUT	Atualizar ou criar um usuário, inserindo dados inválidos.	Deve ser exibida uma mensagem de erro informando os campos que foram preenchidos em um formato inválido.
TU-14	/usuarios/{id}	PUT	Criar um usuário inserindo um ID inexistente e um email já utilizado.	Deverá ser exibida uma mensagem de erro indicando que o email informado já está cadastrado.
TP-01	/produtos	GET	Listar os produtos cadastrados.	A aplicação deve retornar uma lista dos produtos cadastrados.
TP-02	/produtos	POST	Cadastrar um produto com dados válidos.	Cadastrar o produto e exibir uma mensagem de sucesso.
TP-03	/produtos	POST	Cadastrar um produto com dados inválidos.	Deverá ser retornada uma mensagem de erro indicando os campos que foram preenchidos incorretamente.
TP-04	/produtos	POST	Cadastrar um produto com um nome já utilizado.	Deverá ser retornada uma mensagem de erro indicando

				o nome inserido já está sento utilizado.
TP-05	/produtos/{id}	GET	Buscar um produto com um ID válido e existente.	A aplicação deve retornar o produto correspondente ao ID informado.
TP-06	/produtos/{id}	GET	Buscar um produto com um ID inválido.	Exibir uma mensagem de erro informando que o ID informado está fora do formato correto.
TP-07	/produtos/{id}	GET	Buscar um produto com um ID inexistente.	Exibir uma mensagem de erro informando que o ID informado não foi encontrado.
TP-08	/produtos/{id}	DELETE	Excluir um produto com um ID válido e existente.	Retornar uma mensagem de sucesso e excluir o produto correspondente do banco de dados.
TP-09	/produtos/{id}	DELETE	Excluir um produto com um ID inválido.	Exibir uma mensagem de erro informando que o ID informado está fora do formato correto.
TP-10	/produtos/{id}	DELETE	Excluir um produto com um ID inexistente.	Exibir uma mensagem de erro informando que o ID informado não foi encontrado.
TP-11	/produtos/{id}	PUT	Atualizar um produto, inserindo dados corretos e um ID existente.	Os dados do produto devem ser atualizados e retornar uma mensagem de sucesso.
TP-12	/produtos/{id}	PUT	Criar um novo produto, inserindo dados corretos e um ID inexistente.	O novo produto deve ser criado e retornar uma mensagem de sucesso.
TP-13	/produtos/{id}	PUT	Atualizar ou criar um produto, inserindo dados inválidos.	Deve ser exibida uma mensagem de erro informando os campos que

				foram preenchidos em um formato inválido.
TP-14	/produtos/{id}	PUT	Criar um novo produto, inserindo um nome já utilizado e um ID inexistente.	Deverá ser retornada uma mensagem de erro indicando o nome inserido já está sento utilizado.
TC-01	/carrinhos	GET	Listar os carrinhos cadastrados.	A aplicação deve retornar uma lista dos carrinhos cadastrados.
TC-02	/carrinhos	POST	Cadastrar um novo carrinho com dados válidos e token válido.	Cadastrar o carrinho, exibir uma mensagem de sucesso e subtrair a quantidade de produtos inserida dos respectivos objetos em /produtos.
TC-03	/carrinhos	POST	Cadastrar um novo carrinho sem um token válido.	Deve ser retornado uma mensagem de erro de autorização.
TC-04	/carrinhos	POST	Cadastrar um novo carrinho com dados inválidos.	Deve ser exibida uma mensagem de erro informando os campos que foram preenchidos em um formato inválido.
TC-05	/carrinhos	POST	Cadastrar um novo carrinho com um token que já possui um carrinho vinculado.	Retornar uma mensagem de erro informando que o usuário já possui um carrinho cadastrado.
TC-06	/carrinhos/{id}	GET	Buscar um carrinho com um ID válido e existente.	A aplicação deve retornar o carrinho correspondente ao ID informado.
TC-07	/carrinhos/{id}	GET	Buscar um carrinho com um ID inválido.	Exibir uma mensagem de erro informando que o ID

				informado está fora do formato correto.
TC-08	/carrinhos/{id}	GET	Buscar um carrinho com um ID inexistente.	Exibir uma mensagem de erro informando que o ID informado não foi encontrado.
TC-09	/carrinhos/concluir-compra	DELETE	Concluir uma compra com um token válido.	O carrinho deve ser excluído e retornar uma mensagem de sucesso.
TC-10	/carrinhos/concluir-compra	DELETE	Concluir uma compra sem um token válido.	Deve ser retornado uma mensagem de erro de autorização.
TC-11	/carrinhos/cancelar-compra	DELETE	Cancelar uma compra com um token válido.	O carrinho deve ser excluído, a quantidade de cada produto deve ser inserida de volta respectivamente em cada objeto de /produtos.
TC-12	/carrinhos/cancelar-compra	DELETE	Cancelar uma compra sem um token válido.	Deve ser retornado uma mensagem de erro de autorização.

7. Estratégia Exploratória

A estratégia exploratória escolhida foi a “Estratégia dos Tours de James Whittaker”, mais especificamente os tours:

- Do guia turístico: Que segue os caminhos principais e funcionalidades mais utilizadas.
- Do sabotador: Que tenta quebrar o sistema fornecendo dados inválidos e que se esperam gerar um erro.

A estratégia em questão foi escolhida para verificar a responsividade correta da aplicação, se seus principais recursos estão em pleno funcionamento (vender os produtos), a autenticação (para garantir a segurança dos usuários e mitigar prejuízos financeiros) e a precisão da documentação.

Além disso, como serão testes de endpoints utilizando o postman, a estratégia não exige muito da presença de elementos visuais do site nem de selecionar formulários, links, botões e entre outros.

8. Critérios de Entrada

A aplicação estará pronta para ser testada quando os recursos principais para efetuar uma compra e gerenciar o estoque estiverem prontas. Os endpoints /produtos, /usuários, /login e /carrinho devem estar desenvolvidos e possuir a comunicação necessária entre si, afim de que os testes possam avaliar sua eficiência.

A lógica de negócios dos endpoints citados acima também deve estar implementada, tendo em vista que também será analisada de acordo com os objetivos dos testes. Os endpoints dever ter passado pelo menos por um smoke test básico, evidenciados no plano em “Evidências e requisições“, para garantir que o ambiente esteja testável de maneira eficiente e não retorne erros internos do servidor.

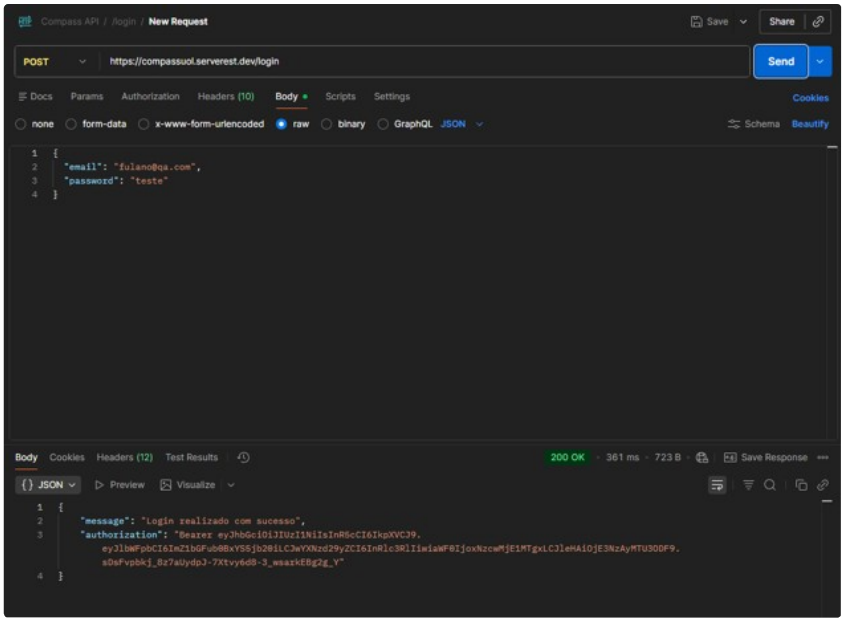
9. Critérios de Saída

A fase de testes será considerada concluída quando todos os cenários de teste tiverem sido executados e seus resultados documentados, os riscos apontados tiverem sido mitigados ou validados, os dados inseridos pelos testes forem devidamente excluídos e o relatório de execução ter sido escrito.

10. Evidências e requisições

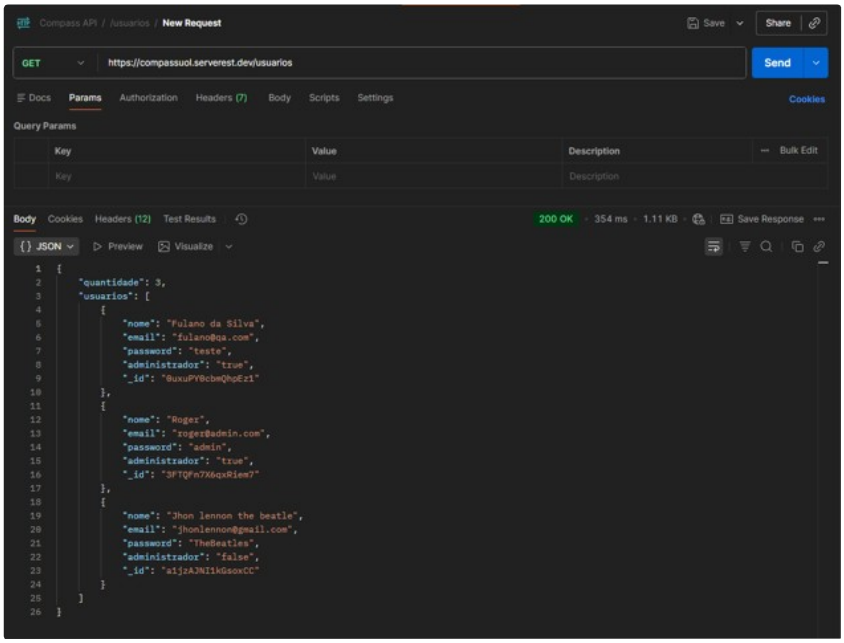
1- /login

POST - Logar com um usuário válido:

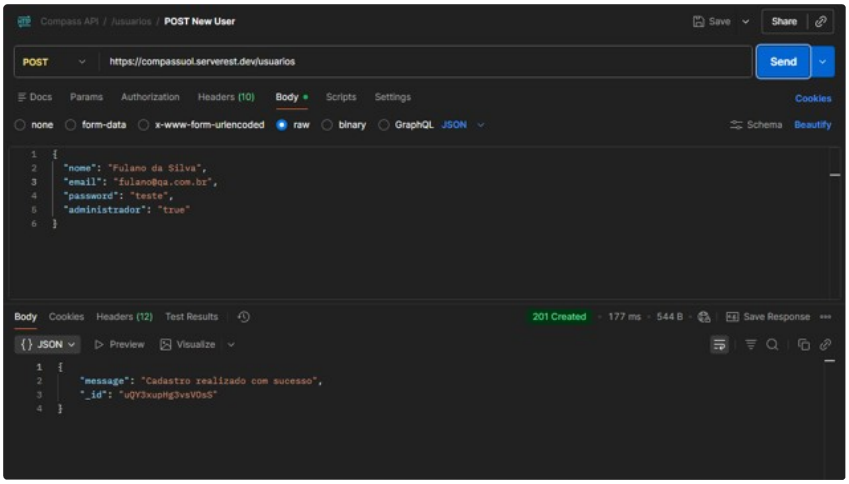


2- /usuarios

GET - Listar os usuários cadastrados:

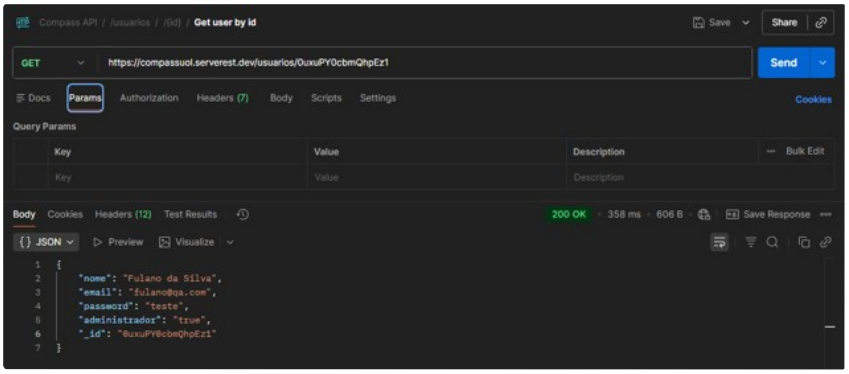


POST - Cadastrar um novo usuário:

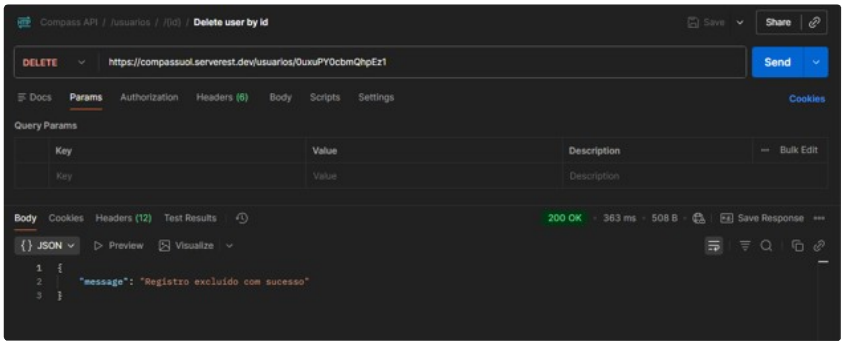


2.1 - /usuarios/{id}

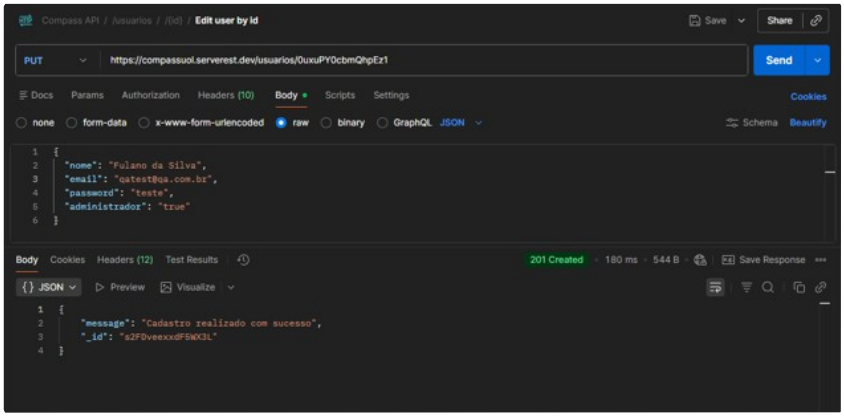
GET - Buscar usuário pelo ID:



DELETE - Deletar usuário pelo ID:

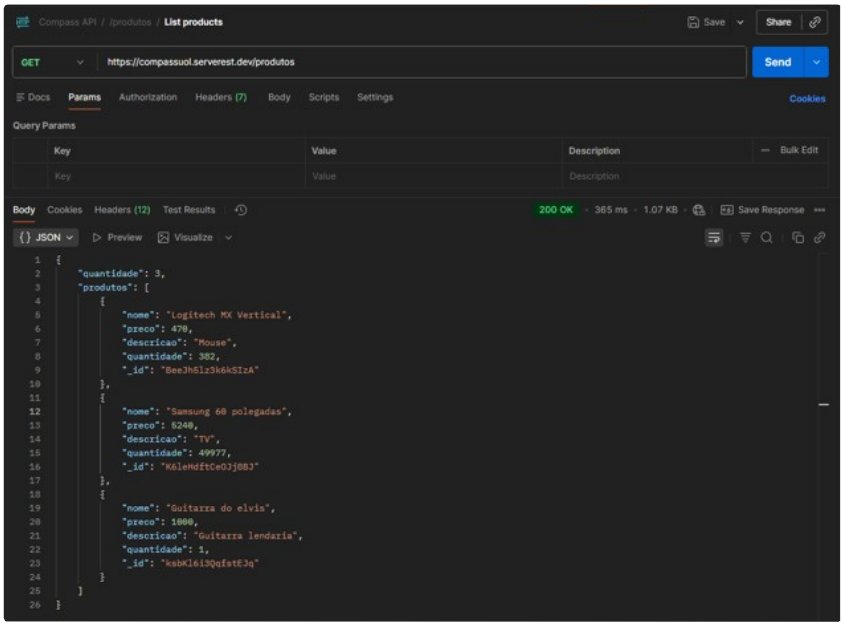


PUT - Editar usuário pelo ID:

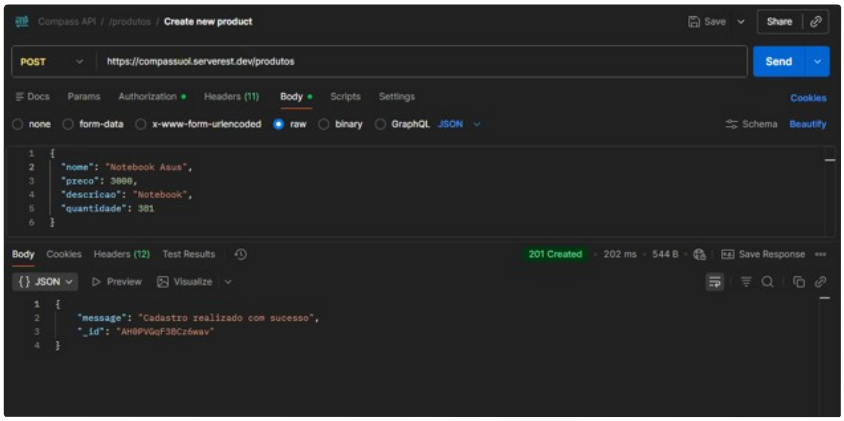


3- /produtos

GET - Listar os produtos cadastrados:

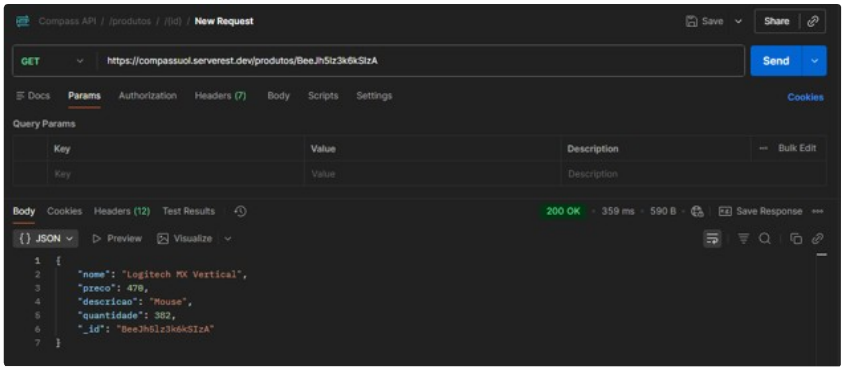


POST - Criar novo produto:

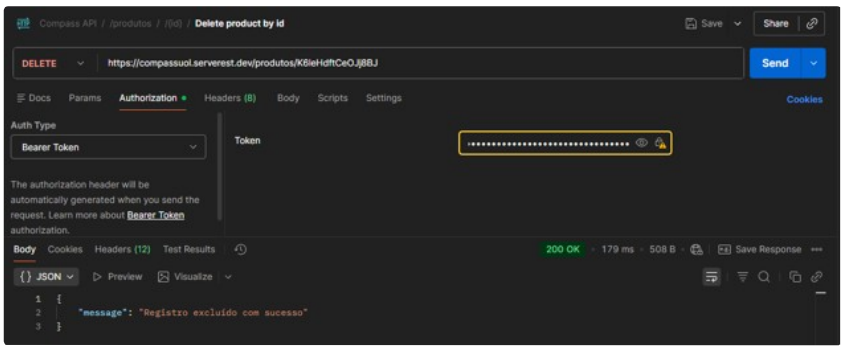


3.1 - /produtos/{id}

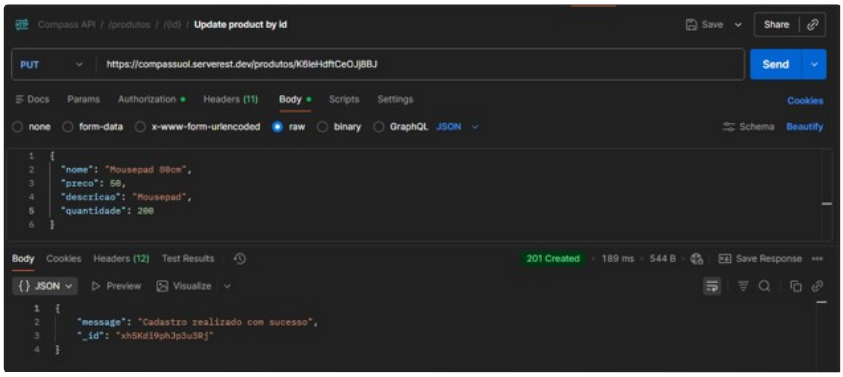
GET - Buscar produto pelo ID:



DELETE - Excluir produto pelo ID:

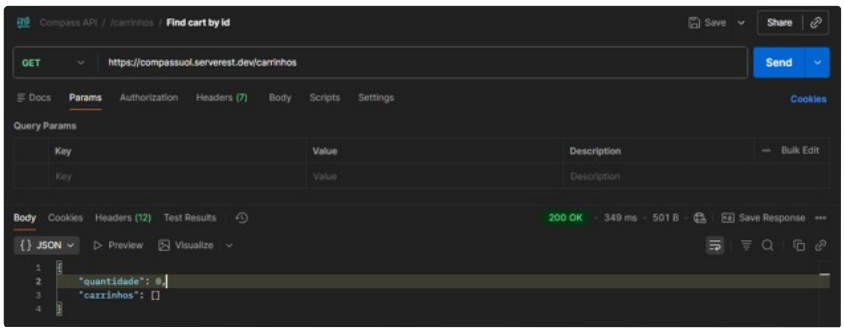


PUT - Editar um produto pelo ID:

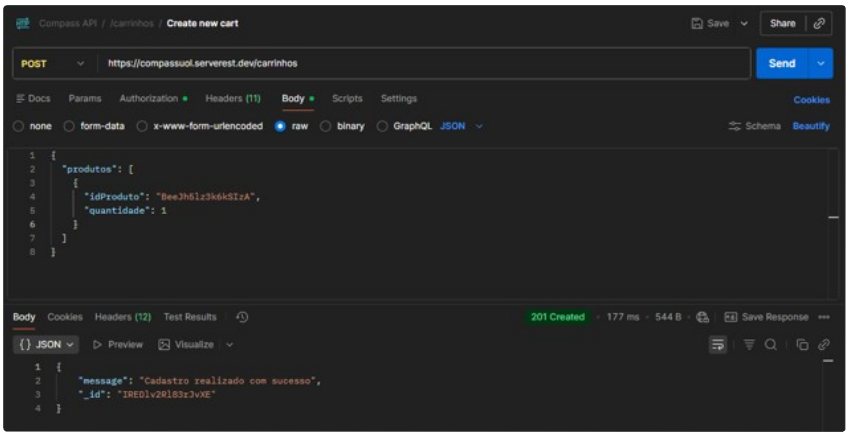


4 - /carrinhos

GET - Listar carrinhos registrados:

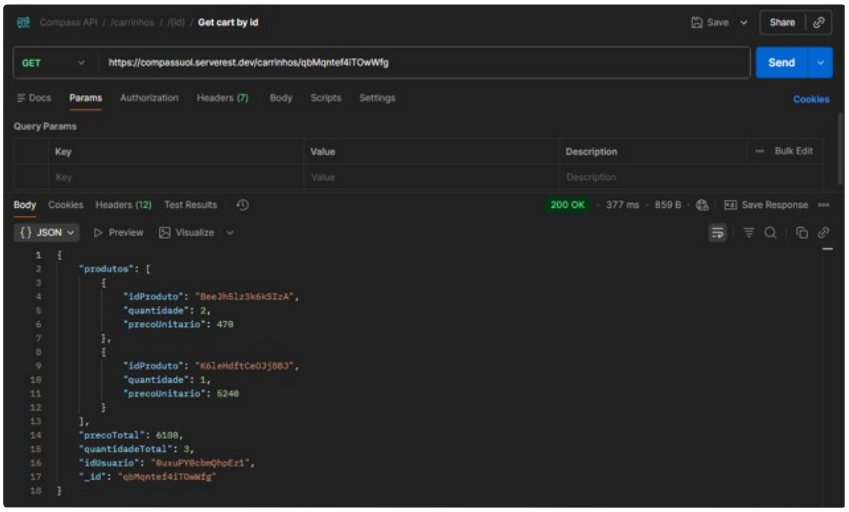


POST - Cadastrar um nov carrinho:



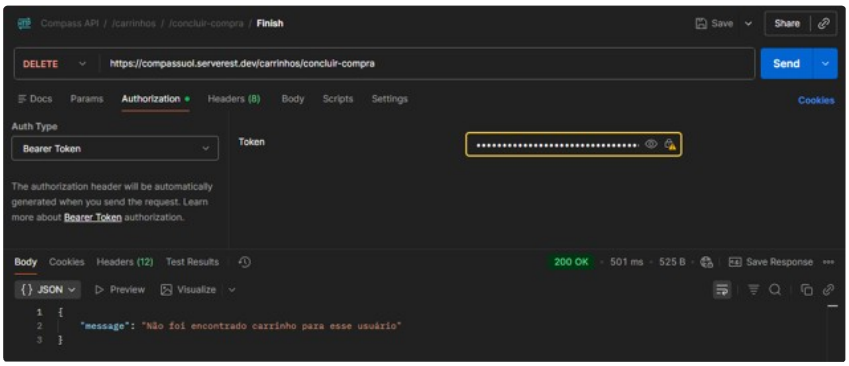
4.1 - /carrinhos/{id}

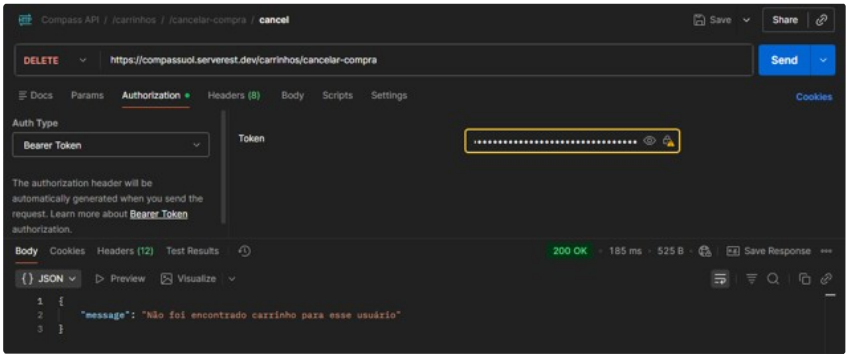
GET - Buscar carrinho pelo ID:



4.2 - /carrinhos/concluir-compra

DELETE - Exclui o carrinho pelo token:



4.3 - /carrinhos/cancelar-compra

DELETE - Exclui o carrinho pelo token e retorna os produtos ao estoque: